

1 | Extending the Probe

This guide provides practical informations for extending the probe. It explains how to add a new capability, configuration group, or configuration parameter while ensuring that the required metadata is included in the announce message and can therefore be displayed correctly in the coordinator web interface.

1.1 Add a new capability

A capability represents a command that can be executed by the probe. Adding a new capability requires three main steps:

1. Implement the command executor.
2. Register the executor.
3. Define and register the capability metadata.

1. Implement the command executor

Create a new `.go` file in the `probe.services` package and implement the function responsible for executing the command.

The function must follow the executor interface used by the probe.

For example:

```
1 func PingExecute(msgCommand models.MsgCommandRequest, p *probe.Probe) 
```

The executor is responsible for processing the command and sending the appropriate response to the coordinator.

Depending on the execution process, it can use the following response functions implemented in the `probe.probe` package:

- **SendSuccessResponse**: sends the final successful result.
- **SendProcessingResponse**: informs the coordinator that the command start being processed.
- **SendErrorResponse**: reports an error during command execution.

For example, an executor for a new `example` capability could be implemented in `example.go` in the function `func ExampleExecute(msgCommand models.MsgCommandRequest, p *probe.Probe)`.

2. Register the executor

Once the executor has been implemented, it must be registered in the Executors map in the `probe.app` package.

The *key* of the map corresponds to the **command name** received by the probe, while the *value* corresponds to the **executor function**.

For example:

```
1 // Register the executors available for processing commands. 
2 Probe.Executors["ping"] = services.PingExecute
```

```

3  Probe.Executors["capture"] = services.CaptureExecute
4  Probe.Executors["portscan"] = services.PortscanExecute
5  Probe.Executors["http"] = services.HTTPExecute
6  Probe.Executors["udp"] = services.UDPExecute
7  Probe.Executors["udp-server"] = services.UDPServerEchoExecute
8  Probe.Executors["tcp"] = services.TCPEExecute
9  Probe.Executors["tcp-server"] = services.TCPServerEchoExecute
10 Probe.Executors["pcap-file"] = services.PcapExecute

```

To add a new capability called `example`, the following entry must be added:

```
1 Probe.Executors["example"] = services.ExampleExecute
```



3. Define the capability metadata

The capability must then be described by creating a new variable of type `models.Capability` structure in the `probe.capabilities` package. This variable defines all the metadata required by the coordinator to display and configure the capability in its web interface.

The variable should follow the naming convention `\<CapabilityName>Capability`. For example, a capability named `Example` should be defined as `ExampleCapability`.

Here is an example of this structure for the ping command:

```

1 // PingCapability fully defines the ping command
2 var PingCapability = models.Capability{
3     Action:      "ping",
4     Label:       "Ping",
5     Icon:        "network_ping",
6     Description: "ICMP echo request test.",
7
8     Params: []models.CapabilityParam{
9         {Key: "interface", Label: "Interface", Type: "string", Example: "eth1"},
10        {Key: "target", Label: "Target", Type: "string", Example: "google.ch"},
11        {Key: "count", Label: "Packet count", Type: "number", Example: 4, Min: 1,
12          Max: 100},
13        {Key: "timeout_ms", Label: "Timeout", Type: "number", Example: 1000, Min:
14          100, Max: 30000},
15    },
16    Expected: []models.Result{
17        {Key: "packets_sent"},
18        {Key: "packets_received"},
19        {Key: "packet_loss_percent"},
20        {Key: "rtt_min_ms"},
21        {Key: "rtt_avg_ms"},
22        {Key: "rtt_max_ms"},
23    },
24 }

```

4. Add the capability to the capability list

Finally, the new capability must be added to `CapabilitiesList` in the `probe.capabilities` package.

For example:

```
1 var CapabilitiesList = []models.Capability{
2     PingCapability,
3     CaptureCapability,
4     PortscanCapability,
5     HTTPCapability,
6     UDPCapability,
7     UDPServerCapability,
8     TCPCapability,
9     TCPServerCapability,
10    PcapCapability,
11
12    ExampleCapability,    // New example capability
13 }
```

Adding the capability to this list ensures that its metadata is included in the probe's **announcement message**.

1.2 Add a New Configuration Group

A configuration group is used to organize related configuration parameters in the coordinator web interface.

Adding a new configuration group only requires adding a new entry to the list returned by the function `SetConfigGroupList()`, implemented in the `probe.probe` package.

1. Add the configuration group

The current configuration groups are defined as follows:

```
1 func (p *Probe) SetConfigGroupList() []models.Group {
2     log.Println("[PROBE] Setting config group")
3     ConfigGroups := []models.Group{
4         {Key: "network", Label: "Network", Icon: "lan"},
5         {Key: "mqtt", Label: "MQTT credentials", Icon: "hub"},
6         {Key: "bridge", Label: "Bridge", Icon: "device_hub", VisibleIf:
7             &models.VisibleCondition{Key: "bridge_mode", Value: true}},
8         {Key: "eth1", Label: "Interface eth1", Icon: "settings_ethernet",
9             VisibleIf: &models.VisibleCondition{Key: "bridge_mode", Value: false}},
10        {Key: "eth2", Label: "Interface eth2", Icon: "settings_ethernet",
11            VisibleIf: &models.VisibleCondition{Key: "bridge_mode", Value: false}},
12    }
13     return ConfigGroups
14 }
```

To add a new group, add another `models.Group` entry.

The group name defined in the *Key* field must be used as the value of the *Group* field when creating a configuration parameter. This associates the parameter with the corresponding configuration group.

1.3 Add a New Configuration Parameter

Adding a configuration parameter requires defining the parameter in the probe configuration, setting its default value, allowing it to be updated, and finally exposing its metadata to the coordinator.

1. Add the parameter to the configuration structure

First, add the new configuration parameter field to `ProbeConfigParams` struct, implemented in the `probe.config` package.

2. Define a default value

Add a default value for the new parameter in the function `SetDefaultConfigParams`, also implemented in the `probe.config` package.

This ensures that the parameter has a valid value when no value is explicitly provided in the configuration file.

3. Allow the parameter to be updated

The new configuration parameter must also be added to the `ValidateControlParams(...)` function, implemented in `probe_HandleControl.go` in the `probe.probe` package.

This function is responsible for validating the parameters received when the coordinator requests a configuration update. The parameter must therefore be explicitly added to the list of accepted parameter names. Otherwise, any update request containing the new parameter will be rejected by the probe with the following error:

Invalid parameter name: ...

In other words, adding the parameter to `ProbeConfigParams` only defines the parameter in the probe configuration. It must also be added to `ValidateControlParams(...)` so that the probe accepts it when receiving configuration updates from the coordinator.

4. Add the parameter metadata

Finally, add the parameter to the function `SetConfigParamsList()`.

This function defines the metadata required by the coordinator to display the parameter and its current value in the web interface.

For example:

```

1 func (p *Probe) SetConfigParamsList() []models.Param {
2     log.Println("[PROBE] Setting config params")
3     Config := []models.Param{
4         {Key: "name", Label: "Name", Type: "string", Value: p.ProbeConfig.Name},
5         {Key: "status_interval_s", Label: "Status interval", Type: "number",
6          Value: p.ProbeConfig.StatusInterval, Min: 1, Max: 3600, Unit: "s"},
7         {Key: "enable", Label: "Enabled", Type: "boolean", Value:
8          p.ProbeConfig.Enabled},
9         {Key: "bridge_mode", Label: "Bridge activated", Type: "boolean", Value:
10          p.ProbeConfig.BridgeMode},
11
12         {Key: "ip", Label: "IP address", Type: "string", Value: p.ProbeConfig.IP,
13          Group: "network"},
14         {Key: "mask", Label: "Subnet mask", Type: "string", Value:
15          p.ProbeConfig.Mask, Group: "network"},
16         {Key: "gateway", Label: "Gateway", Type: "string", Value:
17          p.ProbeConfig.Gateway, Group: "network"},
18         {Key: "subnet", Label: "Subnet", Type: "string", Value:
19          p.ProbeConfig.Subnet, Group: "network"},
20
21         ...
22     }
23     return Config
24 }

```

The *Group* field determines in which configuration group the parameter is displayed.